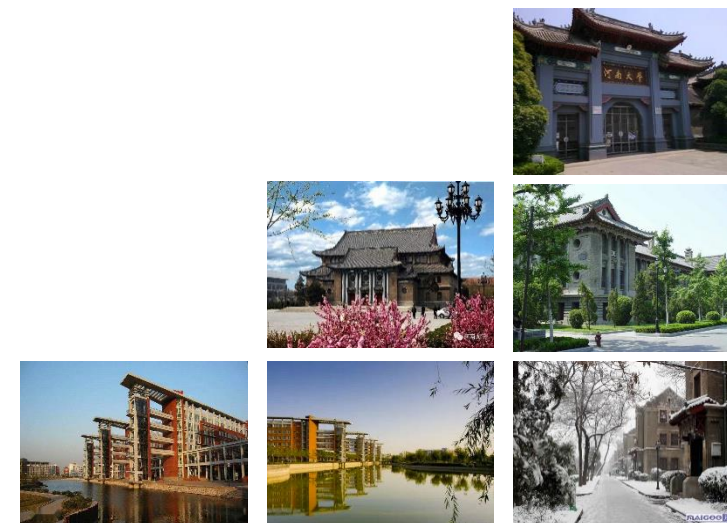




软件工程导论

河南大学计算机与信息工程学院 数据科学系

杜莹



个人介绍

- **姓名：** 杜莹
- **邮件：** `duying@henu.edu.cn`
- **QQ：** 29744930

第一讲课堂安排



第一部分：课前导引

- 为什么学习软件工程
- 软件工程简介
- 课程目标
- 课程安排

为什么学习软件工程？

在实践中，软件开发总会遇到许多问题：

1. 为什么项目总是延期？
2. 为什么软件开发成本如此之高？
3. 为什么我们开发的软件总是不能让用户满意？
4. 为什么用户总是提出新的需求？
5.

学习原因： 软件工程努力为这些问题提供解决方案！

软件工程：

将计算机科学、管理科学等理论运用于指导软件开发与维护的一门交叉学科

课程目标

目标1.掌握软件开发的基本原理、方法；

目标2.掌握软件系统分析、设计、实现、测试能力；

目标3.掌握软件开发的团队协作能力；

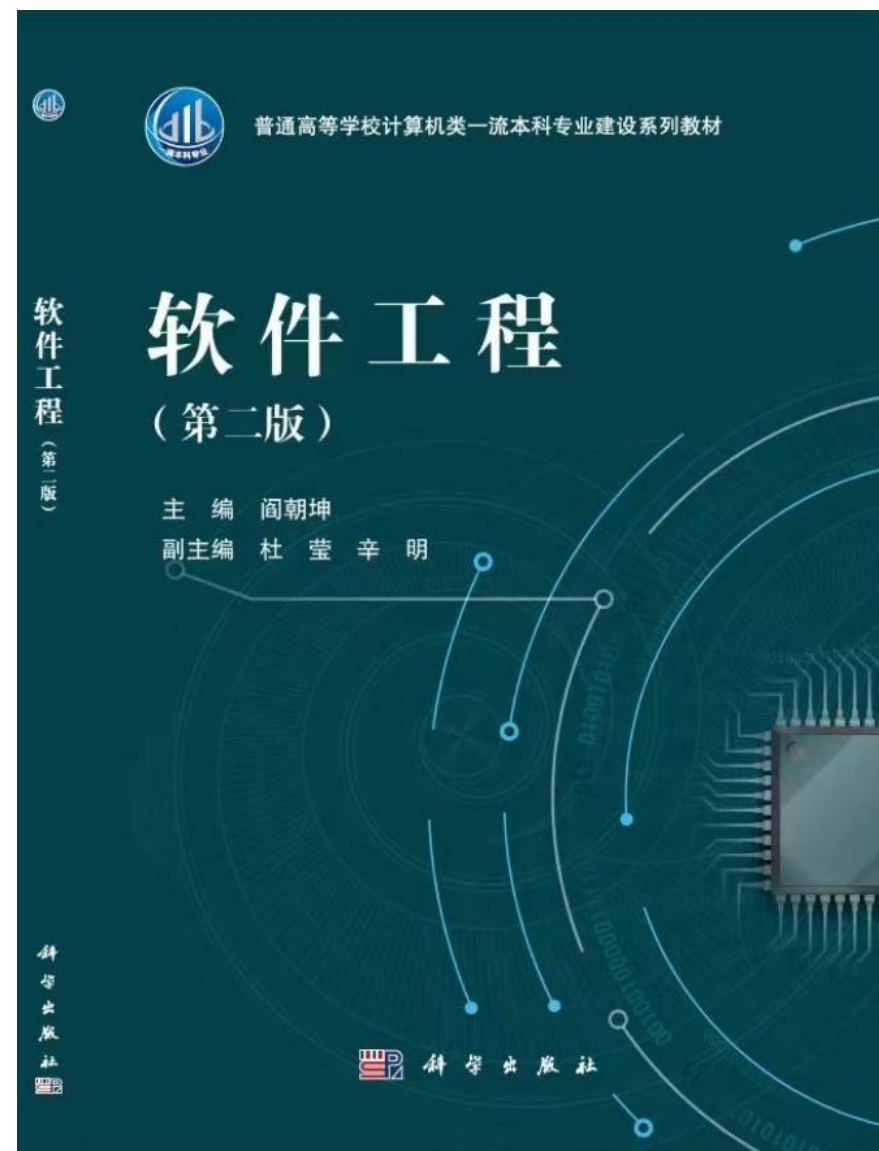
目标4.掌握软件开发过程中的沟通及表达能力；

目标5.掌握软件开发计划、软件项目管理、软件维护的能力。

课程安排-选用教材

- 《软件工程（第二版）》，阎朝坤
杜莹等，科学出版社，2024

内容全面而精简



课程安排-参考书

- **软件工程—理论与实践** 人民邮电出版社 吕云翔（著）
- **软件工程第三版** 陆惠恩，人民邮电出版社
- **软件工程** 钟珞等著，人民邮电出版社
- **软件工程:实践者的研究方法**，普雷斯曼 (Roger S.Pressman) (著), 郑人杰等(译)，机械工业出版社
- **UML用户指南**， 第二版， Grady Booch, James Rumbaugh, Ivar Jacobson(著)， 邵维忠等（译）， 人民邮电出版社
- **设计模式：可复用面向对象软件的基础**， Erich Gamma, Richard Helm ,Ralph Johnson, John Vlissides (著)， 李英军 等（译）， 机械工业出版社
- **Applying use case driven object modeling with UML: an annotated e-commerce example**, Rosenberg D, Scott K., Addison-Wesley

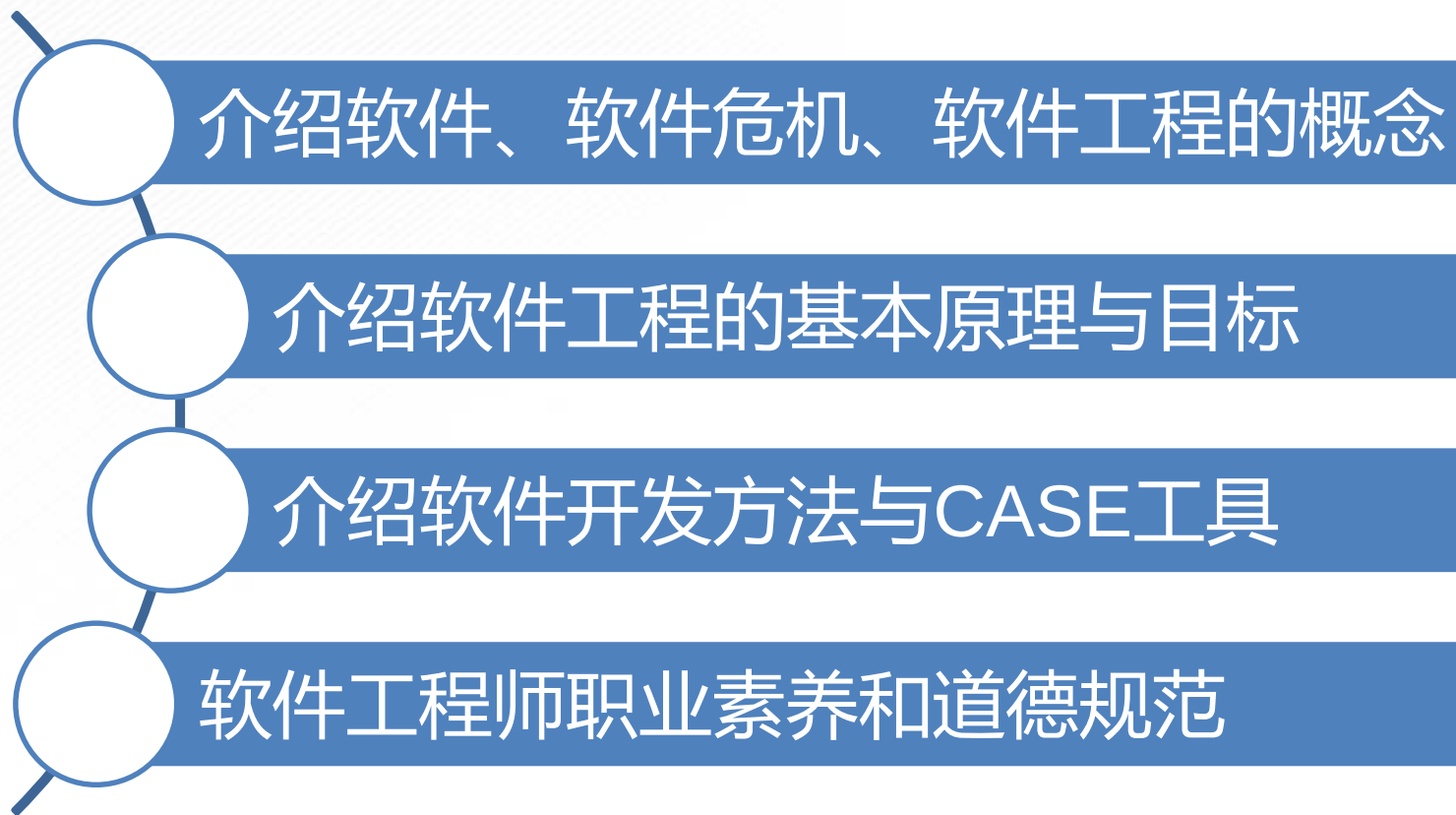
课程进度安排

每次课讲解一个或多个完整主题

进度	授课内容	进度	授课内容
第一讲	课程介绍&软件工程概述	第九讲	面向对象分析（二）
第二讲	软件过程&问题定义	第十讲	面向对象设计
第三讲	需求工程&结构化分析基础	第十一讲	软件体系结构与设计模式
第四讲	结构化分析	第十二讲	用户界面设计与软件实现
第五讲	结构化设计	第十三讲	软件测试与维护
第六讲	面向对象基础	第十四讲	软件项目管理（一）
第七讲	UML建模工具	第十五讲	软件项目管理（二）
第八讲	面向对象分析（一）	第十六讲	软件工程新技术

- 理论与实践相结合
- 以批判发展的眼光看待这门年轻的学科
- 课外阅读为辅助
 - 《人月神话》，布鲁克斯
 - 《浪潮之巅》，吴军
 - 《人件》，Tom DeMarco, Timothy Lister

第二部分：软件工程概述



软件的定义

软件=程序+数据+文档

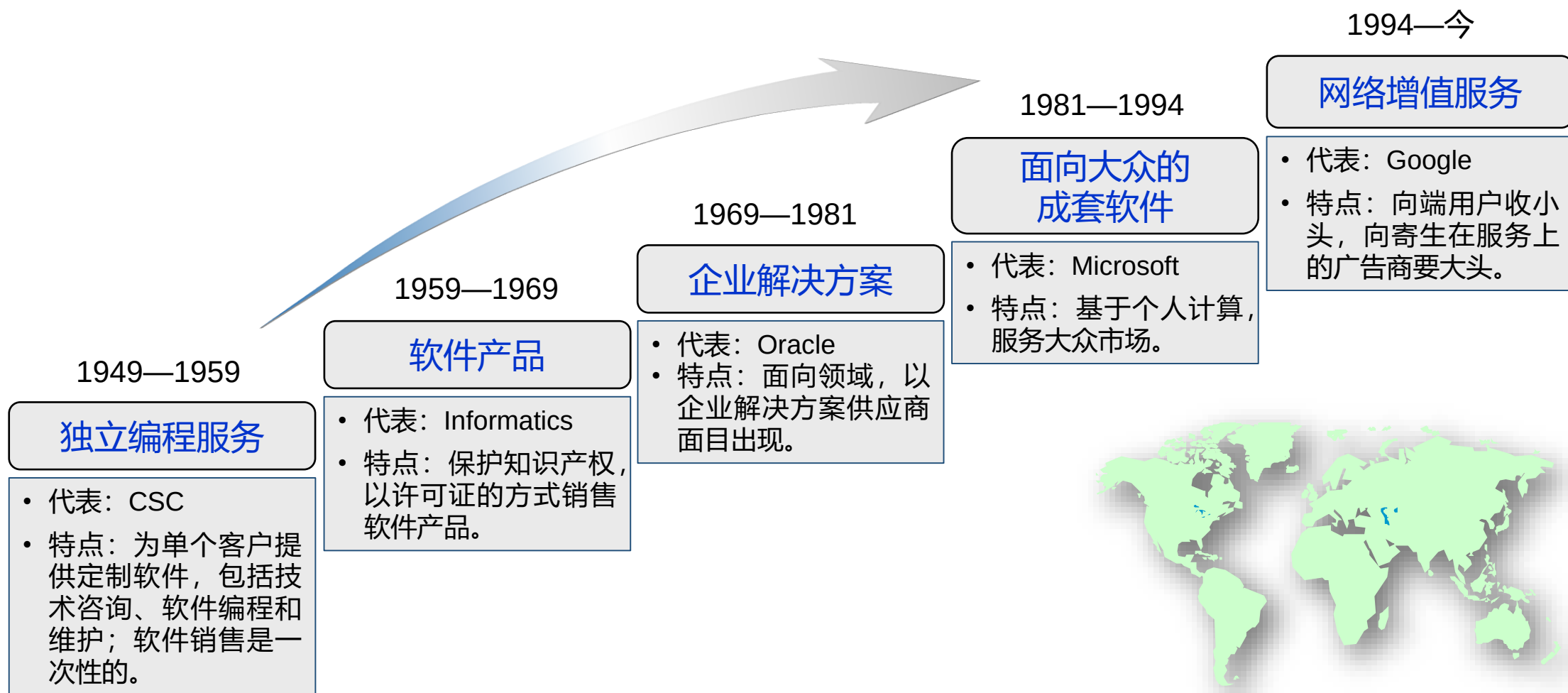
✓ **程序**是按事先设计的功能和性能要求执行的指令序列

程序=算法+数据结构

✓ **数据**是指程序初始化数据、测试数据、以及研发数据、维护数据等

✓ **文档**是与程序开发、维护和使用有关的图文材料

软件发展历程（略）



软件的特点

1. 软件是一种**逻辑实体**，而不是具体的物理实体，因而它具有**抽象性**。
2. 软件是被**开发或设计**的，而不是传统意义上的被制造。
3. 软件**不会磨损**。
4. 软件开发和运行受限于具体系统。
5. 虽然软件产业正在向基于构件的组装前进，大多数软件仍然是**定制**。
6. **软件代替的人类工作，是比较复杂的，成本比较昂贵。**
7. 软件成本比较高。
8. 软件工作会涉及社会因素。

软件分类

按功能划分

系统软件
支撑软件
应用软件

按规模划分

微型、小型、中
型、大型、甚大
型、极大型

按工作方式划分

实时软件、分时
软件、交互式、
批处理

按应用范围划分

通用软件、定制
软件



课程内容

- 1 软件概述
- 2 软件危机
- 3 软件工程



思考

问题1：

以前遇到过的软件失败的案例？

问题2：

IBM360操作系统的开发事件？



软件危机的背景

■ 软件发展的第二个时期

- 软件系统**规模**越来越大
- **复杂程度**越来越高
- **维护难度**越来越大
- 软件**可靠性问题**越来越突出



■ 落后的软件生产方式无法匹配大规模的软件开发需求，导致**软件危机**爆发



软件危机的背景

■ 软件工程时代（1968~至今）

1968年北大西洋公约组织在联邦德国的一次国际学术会议上创造了
软件危机一词。





软件危机的定义

软件在**开发**和**维护**过程中遇到的一系列问题。

- 困扰学术界和工业界
- 用户对软件开发缺乏信心
- 软件开发的高投入和高风险





软件危机的表现

案例

ARIANE 5 火箭

1996年6月4日，Ariane 5火箭在发射37秒之后偏离其飞行路径并突然发生爆炸，当时火箭上载有价值5亿美元的通信卫星。

原因：

- 程序中试图将64位浮点数转换成16位整数时产生溢出
- 缺少对数据溢出的错误处理程序
- 备份软件通过复制而成



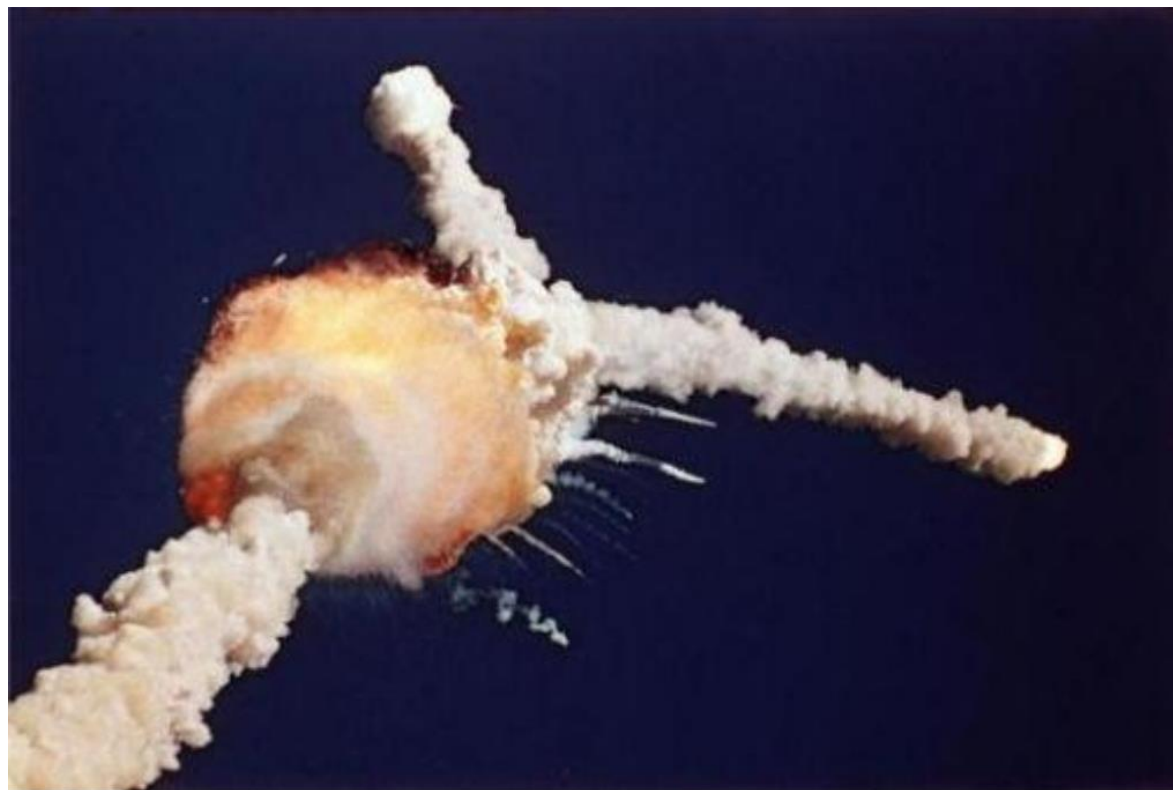


软件危机的表现

案例

【美国火星探测器爆炸事件】

1963年美国用于控制火星探测的计算机软件中的一个“,”被误写成了“.”，致使飞往火星的探测器发生爆炸，造成高达数亿美元的损失。





软件危机的表现

案例

【丹佛新国际机场】

美国丹佛新国际机场自动化行李系统软件，投资1.93亿美元，计划1993年万圣节启用。但开发人员一直为系统错误困扰，屡次推后启用时间，直到1994年6月，机场计划者承认无法预测何时能启用。





软件危机的表现

案例

【德克萨斯癌症中心】

上世纪80年代中期，多名癌症患者在美国东德克萨斯癌症中心接受了计算机控制放射疗法，由于设备therac-25的软件控制系统有问题，放射线严重过量，最后导致死亡。





软件危机的表现

案例

【IBM360 操作系统】

IBM选择的经验丰富的项目经理布鲁克斯，雇佣了最好的系统分析员、编程人员和工程管理人员，投入了5000千人一年的工作量，投资数亿美元，结果还是延期交付，研发费用远远超过了预期，比美国政府研制第一个原子弹的“曼哈顿”计划还要高昂，在项目完成时发现仍然存在大量错误，最终也没实现当初的设想。



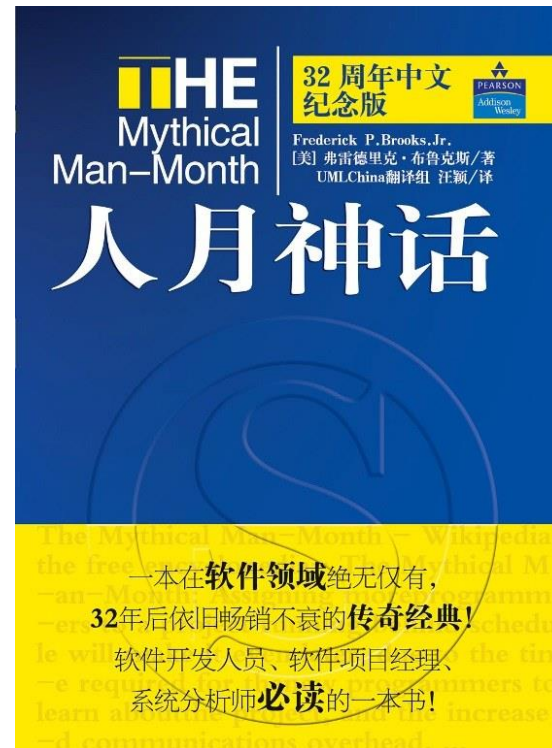


软件危机的表现

案例

【IBM360 操作系统】

传统工业化管理的核心在于将工作划分为简单的步骤，并且优化这些步骤，然后通过管理这些步骤完成产品的过程，而这种方法在软件产品中有很大的局限性，无法很好操作和控制





软件危机的表现

1. 对软件开发成本和进度的估计常常不准确。
2. 用户对“已完成”系统不满意的现象经常发生。
3. 软件产品的质量往往不能得到保证。
4. 软件的可维护程度非常低。
5. 软件通常没有适当的文档资料。
6. 软件成本在计算机系统总成本中所占比例逐年上升。



拉布雷阿的焦油坑

一个值得思考的问题

为什么一个看似简单的东西，却很有可能变成一个落后进度、超出预算、存在大量缺陷的怪物？



Brooks, F. P., *The Mythical Man Month*, Addison-Wesley, 1975

《人月神话》源于Brooks在IBM公司担任OS/360系统项目经理时的实践经验

软件的本质特性



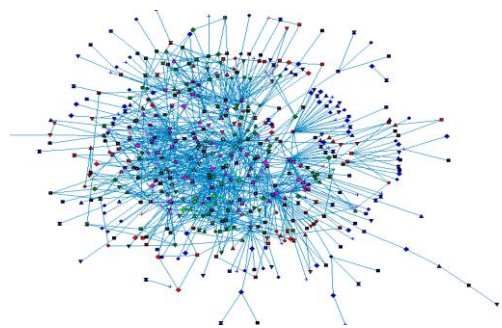
软件具有**复杂度**、**一致性**、**可变性**和**不可见性**等固有的内在特性，这是造成软件开发困难的根本原因

。

Brooks, F. P., "No silver bullet: essence and accidents of software engineering", *IEEE Computer* , Vol. 20, No. 4, pp.10-19, 1987

软件的本质特性：复杂性

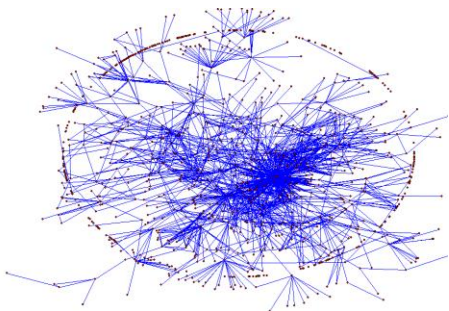
软件是人类思想的外延，人们将自己的思想传送给计算机，当产生的可执行文件被激活运行时，软件便重现人类的意图。



Linux内核有630个函数，存在1814个函数调用。

注：图中将函数表示为节点，调用关系表示为边。

。



Tomcat 软件含1019个类，类与类之间有2109个继承或聚合关系。

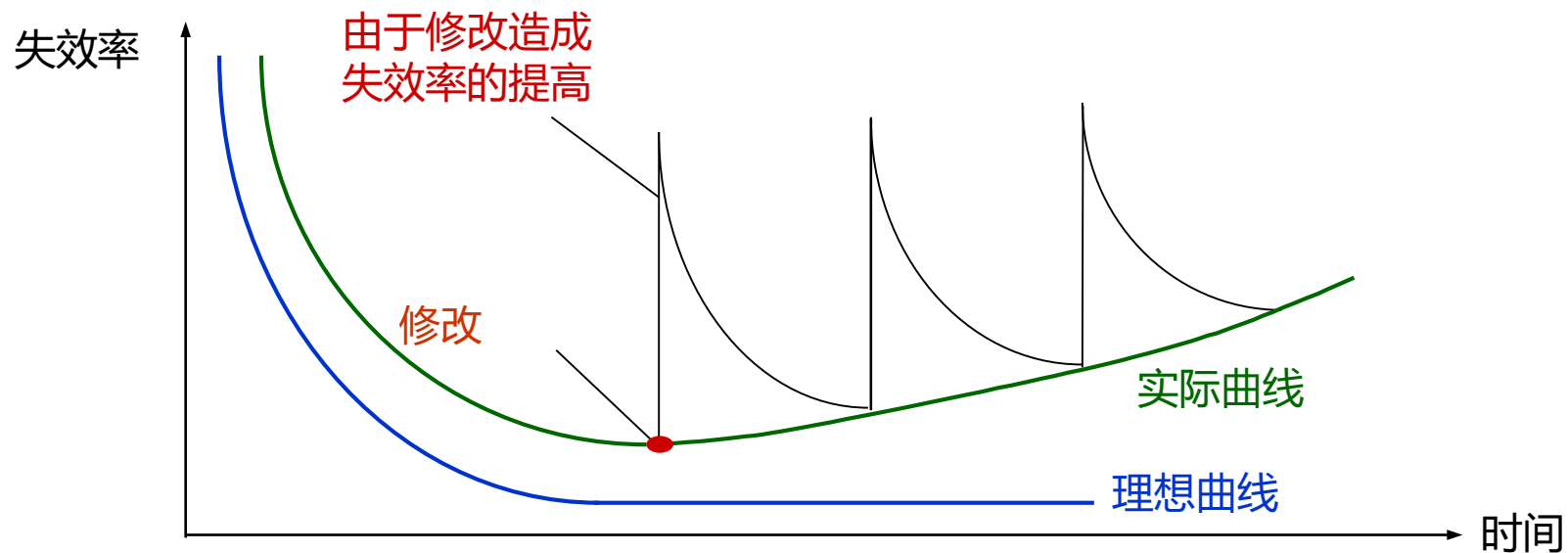
注：图中将类表示为节点，关系表示为边。

软件的本质特性：演化性

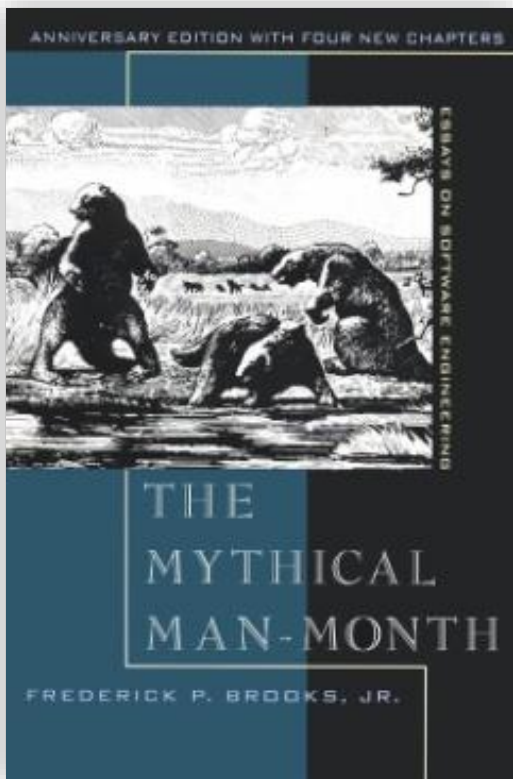
- 人们总是认为软件是容易修改的，但忽视了修改所带来的副作用
- 不断的修改最终导致软件的退化，从而结束其生命周期

eg windows操作系统软件、office等各种软件不仅频繁更换版本更会经常升级

软件的失效率曲线



软件的本质特性：不可见性



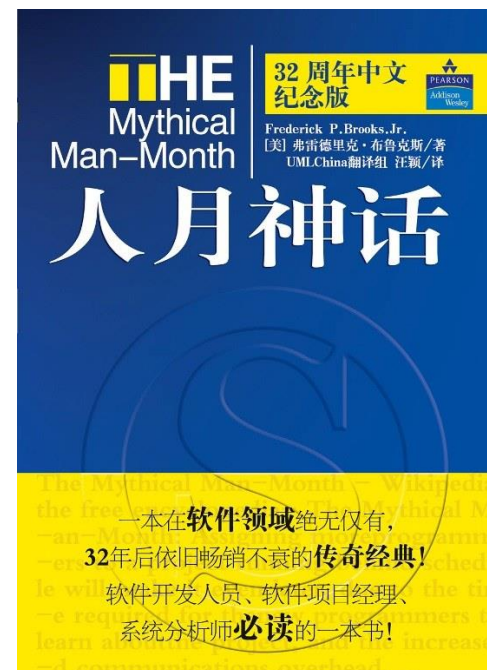
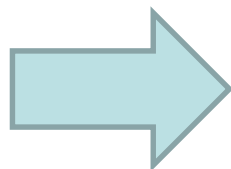
软件人员太像“皇帝的新衣”故事中的裁缝了。当我来检查软件开发工作时，所得到的回答好象对我说：我们正忙于编织这件带有魔法的织物。只要等一会儿，你就会看到这件织物是及其美丽的。但是我什么也看不到，什么也摸不到，也说不出任何一个有关的数字，没有任何办法得到一些信息说明事情确实进行得非常顺利，而且我已经知道许多人最终已经编织了一大堆昂贵的废物而离去，还有不少人最终什么也没有做出来。



“危”与“机”的辩证法



1963年至1966年开发IBM
360机的操作系统。



发现机遇，捕捉机遇，进而创造机遇！



软件危机的原因

- 用户需求不明确。
- 软件开发的规模大。
- 缺乏正确的理论指导。
- 软件复杂度增高。





克服软件危机



技术问题



管理问题

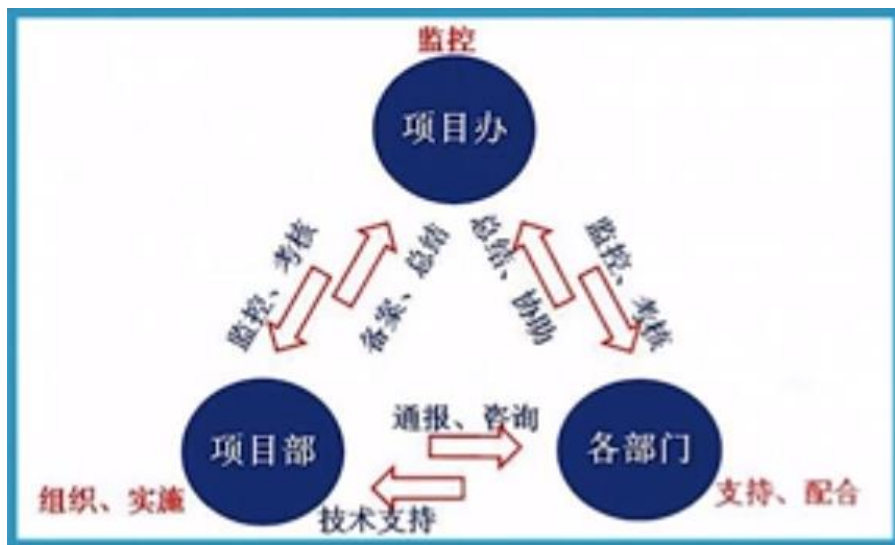
- 软件开发和维护必须走工程化的道路





软件开发和维护必须走工程化的路线

- 吸取和借鉴人类长期以来从事各种工程项目所积累的行之有效的**经验教训、原理方法**
- 采用必要的严密的管理措施做好**项目管理**，还要随时进行**质量检查**。





软件工程

- 实践已经证明，**软件工程**是解决软件危机的最有效方法。
- 软件工程：主要研究软件生产的**客观规律性**。

- 概念
- 原则
- 方法
- 技术
- 工具



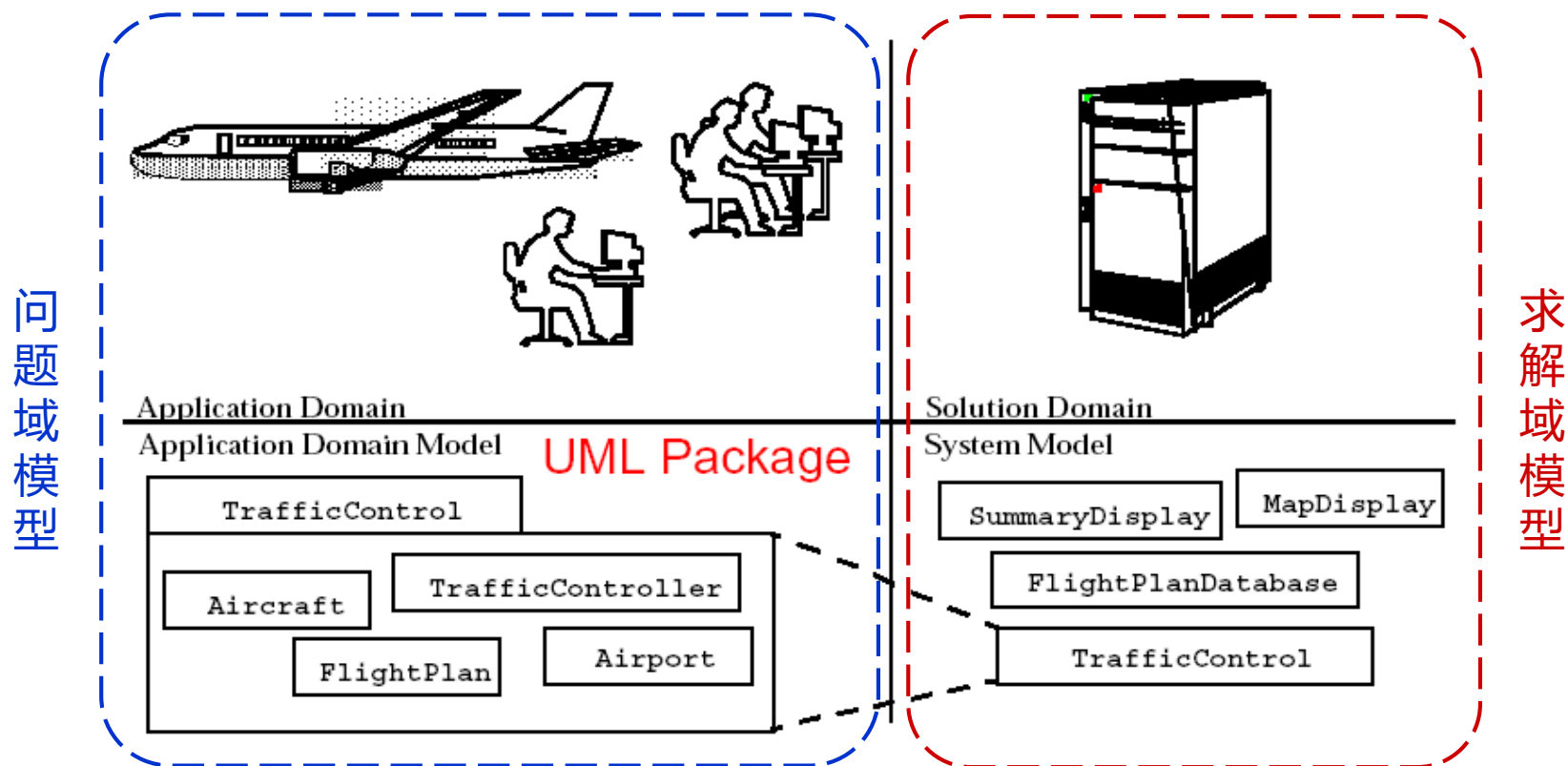
软件危机的主要原因是（ ）。

- ☐ A 软件工程落后
- ☐ B 软件生产能力不足
- ☐ C 实行严格的产品控制
- ☒ D 软件本身的特点及开发方法

提交

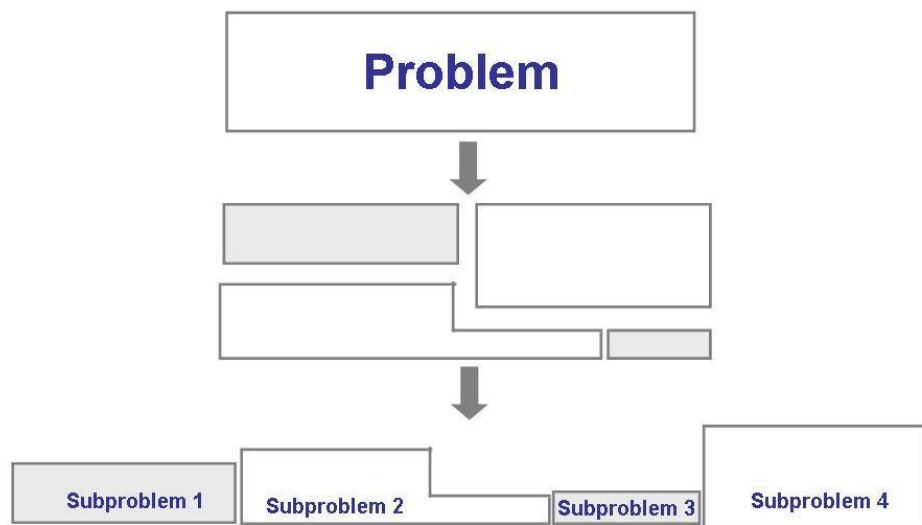
什么是软件工程

软件工程是一项建模活动，通过抽象找到事物的重要特征而忽略非本质的细节，从不同侧面建立系统模型，有效地简化和处理复杂性。

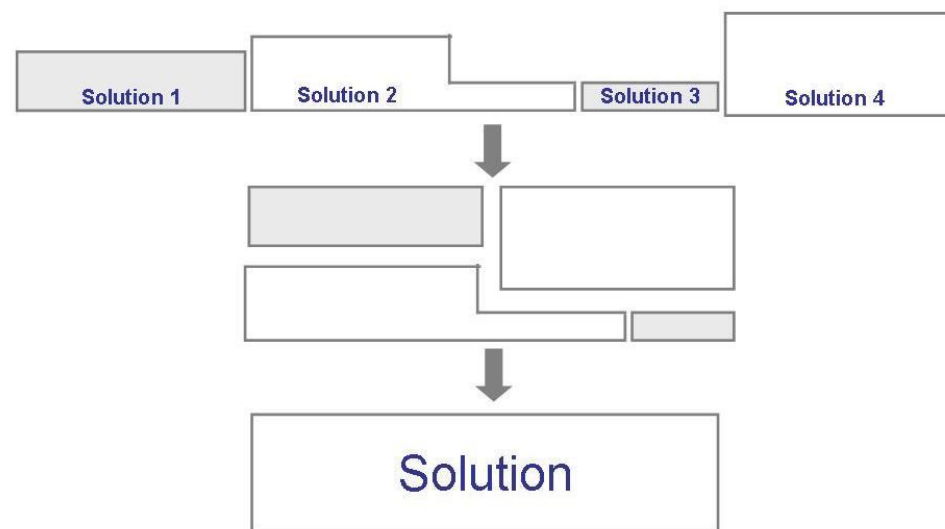


什么是软件工程

软件工程是一项解决问题的工程活动，它不仅限于算法设计，还要通过试验、设计复用、系统评估等手段找到一个客户可接受的方案。



分析：将复杂的问题分解成可理解并能够处理的若干小问题进行研究和分析。



合成：在分析的基础上，针对各个不同部分给出解决方案，并将这些小构造块组合成一个大系统。

什么是软件工程

软件工程是一项受软件工程原理指导的活动，软件工程师需要捕捉和理解一个系统的基本原理模型，并根据评价标准提出合理的开发决策。



举例：千年虫问题

一个基本原理是决策背后的动机：

- 描述要解决的问题（例如年份如何表示？）
- 可选择的解决方案（例如用2个字符还是4个字符表示年份）
- 用于指导决策的准则（例如使内存所占空间最小）
- 软件工程师针对可选的解决方案进行讨论
- 为了解决问题而做出的决策（例如决定用2个字符表示年份）

什么是软件工程

- **软件工程**是为了经济地获得能在实际机器上高效运行的可靠软件，而确立一系列工程原理（方法）。----- Fritz Bauer(弗里茨·鲍尔), NATO, 1968
- **软件工程**是 ① 将系统性的、规范化的、可定量的方法应用于软件的开发、运行和维护，即工程化应用到软件上；② 对①中所述方法的研究。

-----IEEE【IEEE93】



下列说法中正确的是（ ）。

- ☐ A 20世纪50年代提出了软件工程的概念
- ☒ B 20世纪60年代提出了软件工程的概念
- ☐ C 20世纪70年代出现了客户机/服务器技术
- ☐ D 20世纪80年代软件工程学科达到成熟

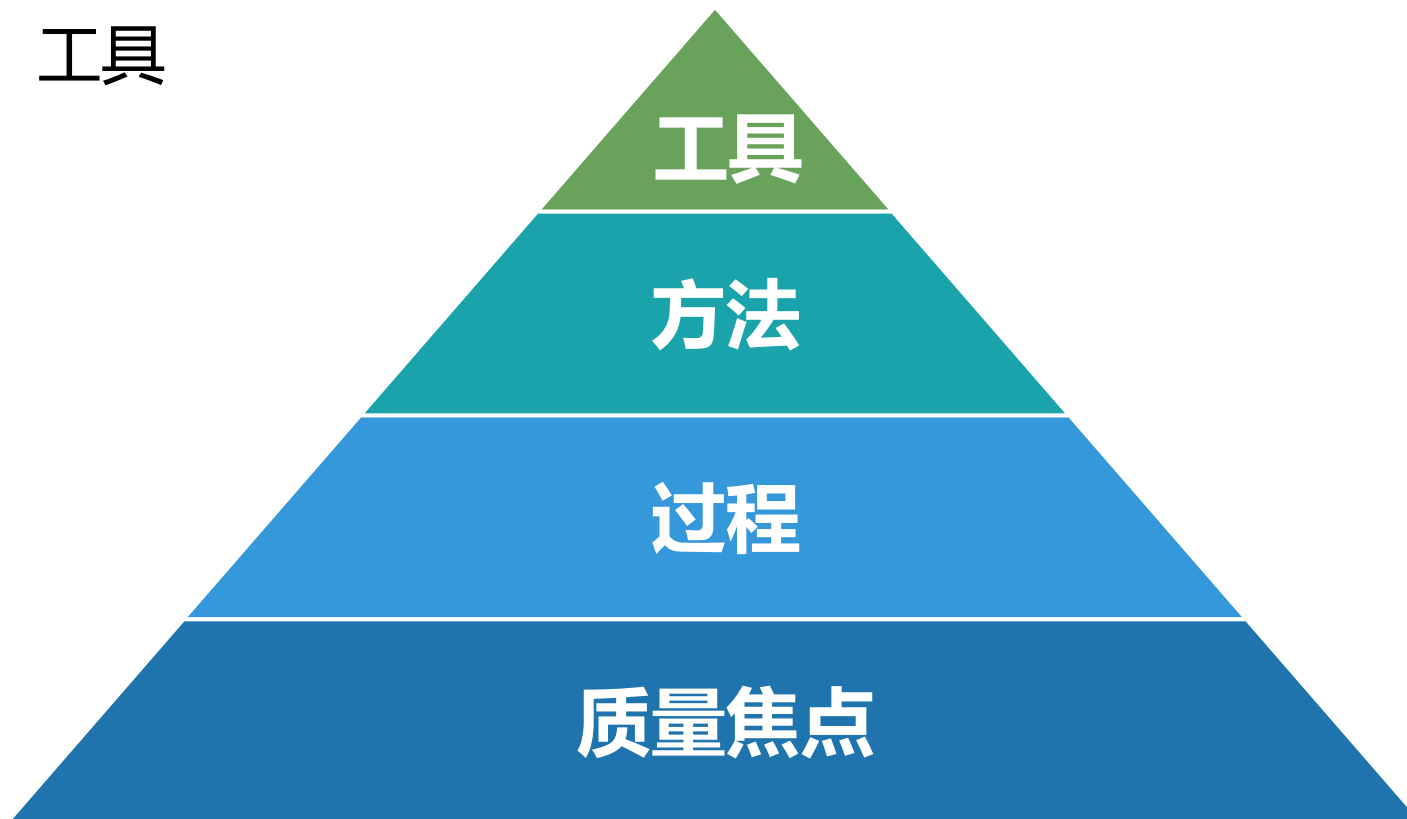
提交

软件工程的研究内容

一个目标： 如何经济的、高质量的开发与维护一个软件系统

三个要素： 过程、方法、工具

重心： 质量焦点

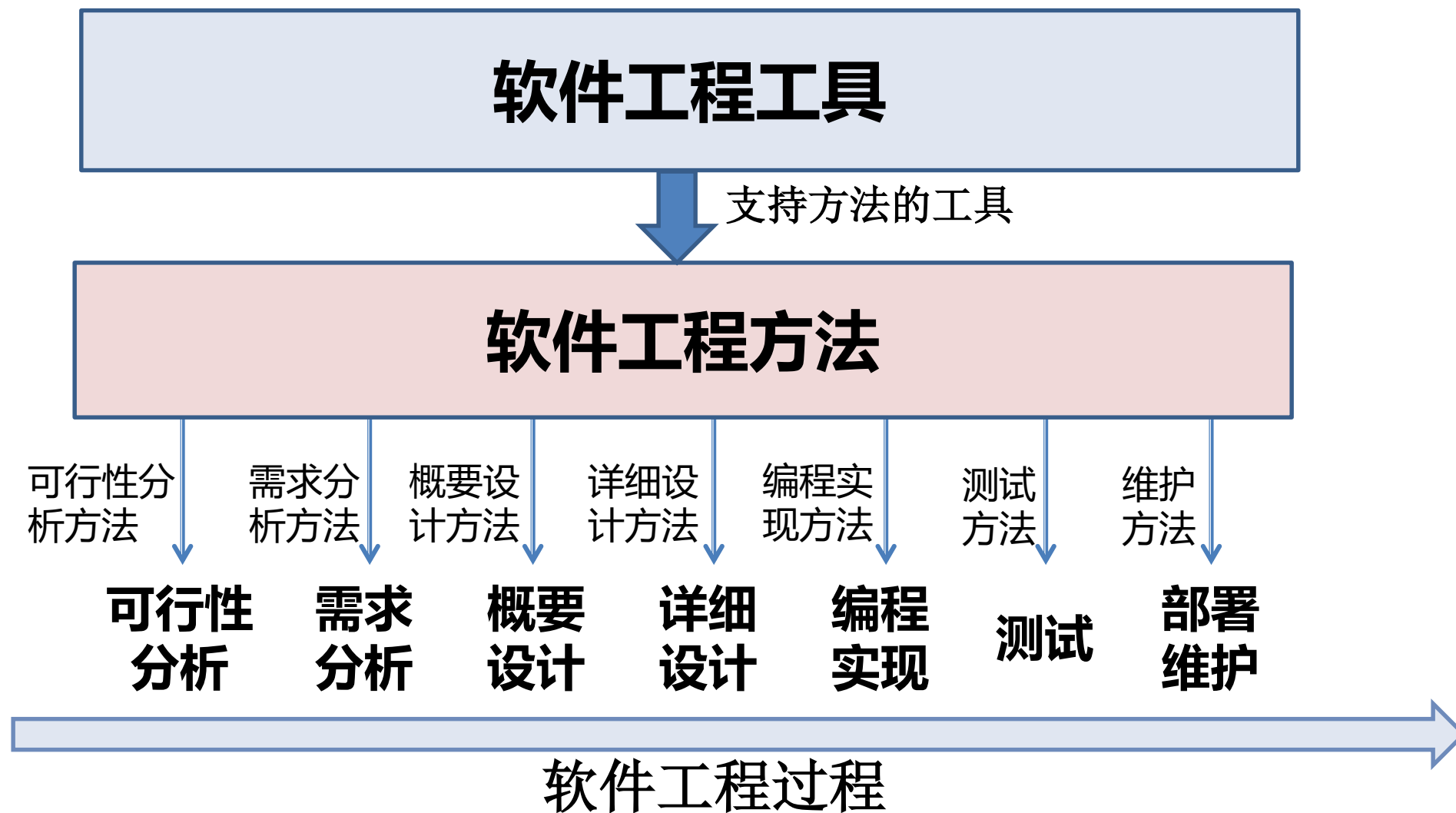


软件工程的三个要素

- ☐ A 技术、方法和工具
- ☐ B 方法、对象和类
- ☒ C 工具、方法和过程
- ☐ D 过程、模型和方法

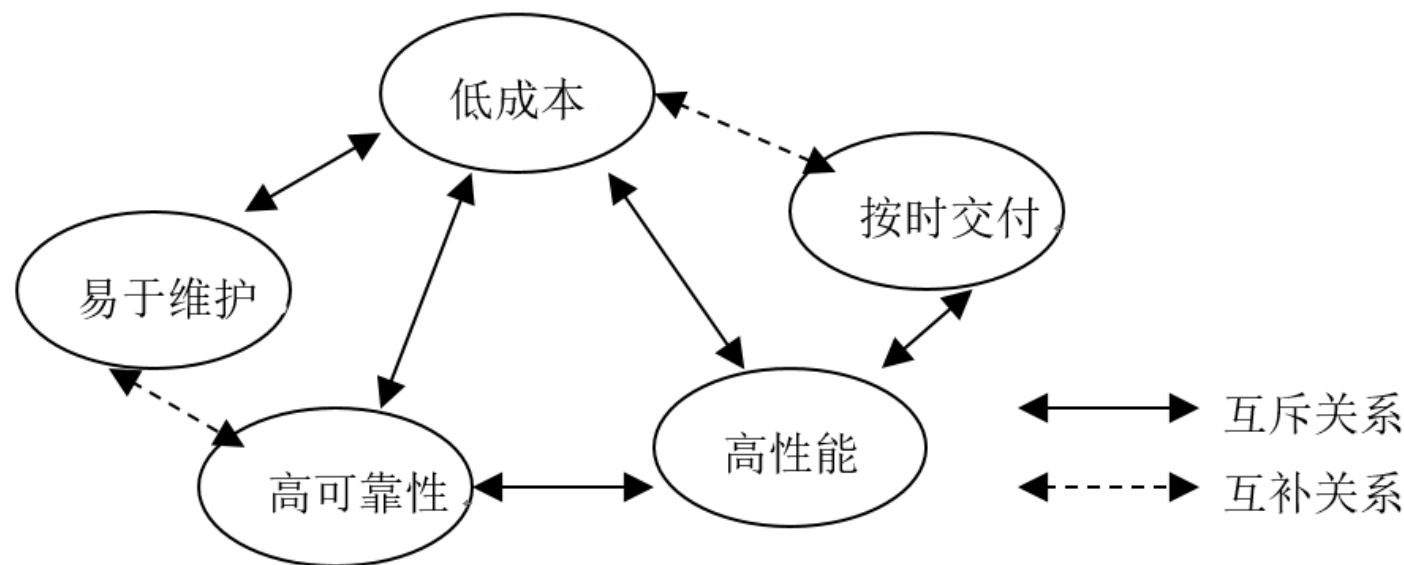
提交

软件工程的研究内容



软件工程的目标

1. 付出较低的开发成本;
2. 取得较高的软件性能;
3. 可靠性高;
4. 易于维护;
5. 能按时完成开发工作, 及时交付使用。



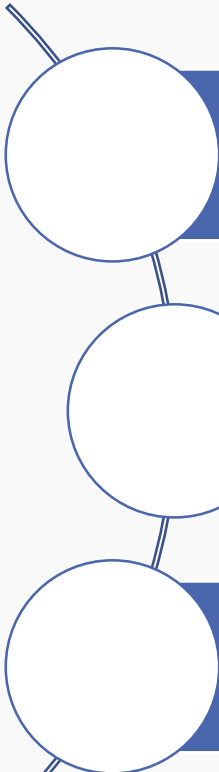
软件工程的原则

- | | |
|----------|---------|
| 1. 抽象; | 5. 确定性; |
| 2. 信息隐蔽; | 6. 一致性 |
| 3. 模块化; | 7. 完备性 |
| 4. 局部化; | 8. 可验证性 |

软件工程的基本原理

1. 用分阶段的生命周期计划严格管理。
2. 坚持进行阶段评审。
3. 实行严格的产品控制。
4. 采纳现代程序设计技术。
5. 结果应能清楚地审查。
6. 开发小组的人员应少而精。
7. 承认不断改进软件工程实践的必要性。

软件工程中的开发方法又称为软件工程方法论，是指软件开发过程中所应遵循的方法和步骤。



1 结构化的开发方法

2 面向数据结构的开发方法

3 面向对象的方法

1 结构化开发方法



结构化方法也称为面向过程的软件开发方法或Yourdon方法，是由E.Yourdon和L.Constantine提出的，是20世纪80年代使用最广泛的软件开发方法。

按照功能分解的原则，自顶向下、逐步求精，直到实现软件功能为止。

使用的工具有：数据流图、数据字典、问题描述语言、判定表和判定树等。

2 面向数据结构的开发方法

面向数据结构的软件开发方法有两种：

1. 1974年由J.D.Warnier提出的结构化数据系统开发方法(Data Structured System Development, DSSD), 又称Warnier方法;
2. 1975年由M.A.Jackson提出的Jackson系统开发方法(Jackson Systems Development, JSD), 简称Jackson方法。

基本思想是：从目标系统的输入/输出数据结构入手，导出程序的基本框架结构，在此基础上，对细节进行设计，得到完整的程序结构图。

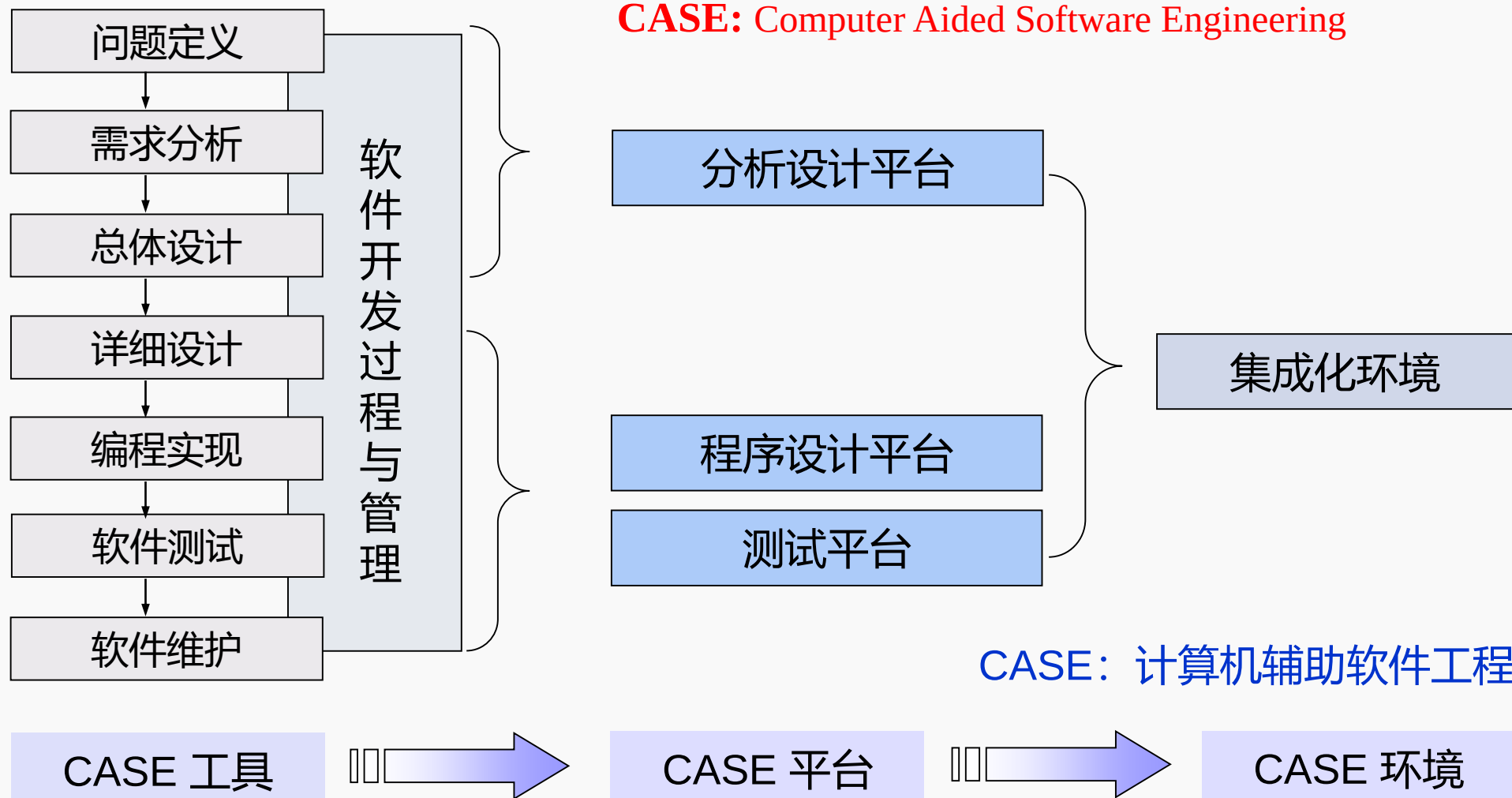
3 面向对象的方法

面向对象方法符合人类的思维方式。它通过将现实世界问题向面向对象解空间映射的方式，实现对现实世界的直接模拟。

20世纪90年代以来，出现了很多种面向对象的分析或设计方法，比较流行的有十几种。这些方法中，有较大影响的有：Booch方法、OMT方法、OOSE方法、Coad/Yourdon方法和UML方法。

面向对象方法是最流行的方法。

CASE工具



软件工程师职业素养



软件工程师是从事软件开发相关工作的人员的统称，包括软件设计人员、软件架构人员、软件工程管理人员、程序员等一系列岗位。

软件工程师不仅需要扎实的计算机基础理论知识，还应具有以下业务能力：

1. 规范化、标准化的编码能力
2. 认识和运用数据库的能力
3. 较强的动手能力和解决实际问题的能力
4. 持续的学习能力，掌握最新的IT实用技术
5. 较强的英语阅读和写作能力
6. 较强的沟通表达能力

软件工程师应致力于使软件的分析、规范、设计、开发、测试和维护成为一种有益和受人尊敬的职业。根据对公众健康、安全和福利的承诺，软件工程师应从以下方面遵守各自的原则：

- | | |
|-----------|--------|
| 1. 产品； | 5. 管理； |
| 2. 公众； | 6. 职业； |
| 3. 判断； | 7. 同事； |
| 4. 客户和雇主； | 8. 本人。 |

本章小结

1. 软件的有关概念，包括软件的定义、特点，并从不同的角度对软件进行分类；
2. 软件危机、软件工程的概念；
3. 软件开发中的三种典型的方法及CASE工具；
4. 软件工程师应具备的职业素养和道德规范。

